

AutoResearcher2: Research Progress Memo

v4.8 — Generator-Critic Pipeline

autoresearcher2 (autonomous)

2026-03-15 20:28

Executive Summary

An autonomous research agent, starting with zero domain knowledge, designed and executed **77** GPU experiments across 2 projects, costing **€0.20** in electricity (0.715 kWh).

Through a generator-critic pipeline with an evolving world model, the system developed **49 explicit beliefs** about how training parameters interact — each with a confidence score, supporting evidence, and natural language explanation.

All results are exploratory (single-seed, no repetitions). The value is not in any individual finding, but in the agent's ability to build and communicate a structured understanding autonomously.

Source: <https://github.com/erikdebruijn/autoresearcher2>

Glossary

val_bpb — validation bits-per-byte; lower = better language model.

mean_reward — average game score; higher = better agent.

DEPTH — number of transformer layers (model capacity).

MATRIX_LR — learning rate for the main weight matrices.

WEIGHT_DECAY — regularization strength (prevents overfitting).

Confidence — heuristic 0–1 score assigned by the LLM based on evidence count and consistency. Not a calibrated probability; treat as a ranking signal.

Belief — an LLM-generated claim about the domain, updated after each experiment. May mix observation and inference.

Tension — an open question the agent has identified but not yet resolved.

1 Projects

1.1 NanoGPT

Target: minimize val_bpb

Runs: 51 successful, 20 failed

World model: v102 — 40 beliefs, 11 tensions

Cost: 0.671 kWh, €0.18, 4.6 hours wall time

Best result

val_bpb = **1.036205** at run 49 (config_change)

Config: DEPTH=8, MATRIX_LR=0.03,
WEIGHT_DECAY=0.1, batch_size=64

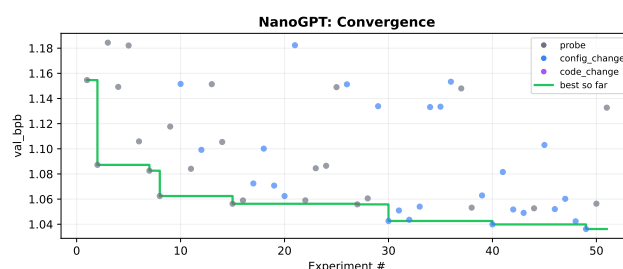


Figure 1: NanoGPT: convergence over 51 experiments. Green line = cumulative best.

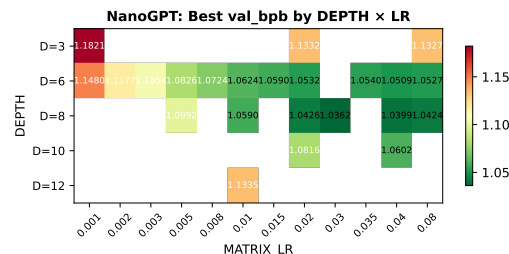


Figure 2: Best val_bpb per DEPTH × MATRIX_LR cell. Lower (green) is better.

Key beliefs

- [0.99] The training system runs beyond the configured num_steps (1000 requested, 2395 actual), likely completing full epochs over the data
- [0.99] The training infrastructure has a fragile dependency on a cached tokenizer file at /home/autoresearcher/.cache/autoresearch/tokenizer/tokenizer.pkl. If this file is missing or the
- [0.98] Peak VRAM for DEPTH=6 is ~14 GB regardless of whether batch_size is 64 or 128, suggesting model parameters dominate memory usage over batch size in this range
- [0.97] Peak VRAM for DEPTH=8 with batch_size=64 is ~23.5 GB, roughly 1.7x the ~14 GB used by DEPTH=6 configurations

5. **[0.97]** The training system processes ~131K tokens per step regardless of batch_size (64 or 128) and depth (6 or 8), suggesting a fixed token-per-step budget. This explains why deeper/slow

... and 35 more beliefs

Open questions

- Only one data point establishes the depth-6 baseline; unclear whether this is near-optimal or far from optimal for this param count and token budget
- The num_steps parameter appears ignored by the training system (400 requested, 2463 run). If this is epoch-based, the 'num_steps' intervention spec may not control training duration
- DEPTH=8 (67.6M params) achieves val_bpb=1.099, which is WORSE than the best DEPTH=6 result (35.8M params, val_bpb=1.084) despite nearly 2x parameters. However, this comparison is h

Measured cost per experiment

Type	Wall time (s)	Energy (Wh)	Cost (€)
config_change	320	29.8	0.0086
probe	320	29.1	0.0084

1.2 Atari Breakout

Target: maximize mean_reward

Runs: 6 successful, 0 failed

World model: v97 — 9 beliefs, 2 tensions

Cost: 0.045 kWh, €0.01, 0.4 hours wall time

Best result

mean_reward = **3.600000** at run 2 (probe)

Config: total_timesteps=50000,

learning_rate=0.00025, n_envs=4

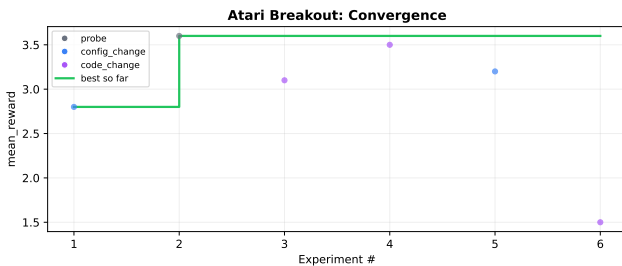


Figure 3: Atari Breakout: convergence over 6 experiments. Green line = cumulative best.

Key beliefs

1. **[0.75]** The agent is in early training explained_variance rose from 0.758 to 0.845 near end of run, suggesting the value function is still improving

2. **[0.60]** Entropy loss decreased from -0.282 to -0.267 over final iterations, indicating policy is still exploring but gradually specializing
3. **[0.55]** PPO with lr=0.00025, 4 envs, 200k timesteps yields mean_reward=2.8 (std=1.6) on this Atari task
4. **[0.50]** The lr decay schedule produces substantially lower entropy (-0.21) and very low clip_fraction (~0.004-0.007), indicating a more converged but potentially prematurely specialized po
5. **[0.50]** Reward shaping (survival_bonus=0.01, life_loss_penalty=-1.0) with PPO at lr=2.5e-4 and 200k timesteps yields mean_reward=1.5 (std=1.2), substantially worse than the unshaped baseli

... and 4 more beliefs

Open questions

- The lr decay schedule (1e-3→1e-5) achieved mean_reward=3.5 vs fixed lr=2.5e-4's 2.8. However, the decay schedule also drove entropy much lower (-0.21 vs -0.267) and clip_fraction n
- Reward shaping (survival bonus + life-loss penalty) was intended to accelerate early learning but reduced mean_reward from 2.8 to 1.5 a 46% drop. The value function learned the s

Measured cost per experiment

Type	Wall time (s)	Energy (Wh)	Cost (€)
config_change_200k_steps	252	7.0	0.00
reward_shaping_200k_steps	271	7.1	0.00

2 Method

Pipeline

The system follows an OODA loop: **Orient** (update world model from latest result) → **Generate** (LLM proposes 5 experiments, reasoning from epistemic intent) → **Critique** (LLM ranks by information value, selects top 3) → **Execute** (run on GPU, parse metrics, feed back).

Each proposal follows: epistemic intent → rationale → expected learning → intervention → cost estimate. This ensures experiments are designed to *learn something regardless of outcome*.

Experiment protocol

Each run trains a single model configuration to completion (single seed, no repetition). The agent controls hyperparameters (config_change) or full training scripts (code_change). A run is “successful” if it completes without error and produces a metric; “failed” runs (OOM, script errors) still inform the world model. Energy consumption is measured per-GPU via nvidia-smi polling at 1 Hz.

Confidence scores

Confidence values (0–1) are assigned by the LLM during world-model updates. They reflect evidence count and internal consistency, not frequentist probabilities. They are useful for ranking beliefs but should not be interpreted as calibrated.

3 Limitations

- **Single seed:** no statistical significance; any result could be noise
- **No ablation:** the LLM’s contribution vs. simpler optimization is untested
- **Hardware-specific:** all runs on one GPU (RTX PRO 6000, 96 GB VRAM)
- **Beliefs mix observation and inference:** claims like “X explains Y” are the agent’s interpretation, not proven causal links
- **Exploratory:** this memo reports what the agent found, not what is established